

COMPSCI 1JC3
Introduction to Computational Thinking
Fall 2026

Haskell Programming Style

Dr. William M. Farmer
McMaster University

Revised: August 25, 2026

Programming is as much an art as it is a science. There is no best way of writing programs. Each programmer needs to develop their own style. Different programming languages require different programming styles. In this short document, we will present rules and recommendations for programming in Haskell. You are required to follow the rules below that are stated in boldface.

1 General Recommendations

You should strive to write code that is readily understandable and easy to maintain and reuse. **Keep your lines of code relatively short, no more than 80 characters long.**

Indentation

Indentation is a required part of Haskell syntax. **Always indent code blocks the same number of spaces, preferably 2 or 4.** Do not indent with tabs.

2 White Space

Use white space consistently. **Put a blank line before each new top-level definition.** You should usually put a space before and after each infix function.

3 Comments

Put a comment before each top-level definition that describes what is being defined. Otherwise keep your comments to a minimum and use them only to point out things that may not be obvious to the reader. Write your code so that it speaks for itself.

4 Names

Give Haskell objects descriptive names. The greater the scope of the object, the more specific should be its name. So the name of a top-level function should be very specific, while the names of its inputs can be single letters. The names of variables and functions should start with a lower-case letter, while the names of types, constructors, and type classes should start with an upper-case letter. Distinguish words in a name using capitalization, so write the name `startPoint` instead of `startpoint`.

5 Function Definition

Each top-level function definition must have the following three components: a descriptive comment, a type declaration, and the definition itself. Here is an example:

```
{-
- This function computes the 2-dimensional norm.
-}
norm :: Float -> Float -> Float
norm x y = sqrt ((x ** 2) + (y ** 2))
```

6 Let and Where Expressions

Use let and where expressions to make expressions more concise and perspicuous. You should usually use a let or where expression to name any compound expression that appears more than once in an expression. Here are examples of how the `norm` function from above can be defined using let and where:

```
norm1 :: Float -> Float -> Float
norm1 x y =
  let
    sq a = a ** 2
    x'   = sq x
    y'   = sq y
  in sqrt (x' + y')
```

```
norm2 :: Float -> Float -> Float
norm2 x y = sqrt (x' + y')
  where
    sq a = a ** 2
    x'   = sq x
    y'   = sq y
```

7 Conditional, Guarded, and Case Expressions

Use conditional expressions, guarded function definitions, and case expressions carefully. Avoided nesting these kinds of expressions.

8 Pattern Matching

Use pattern matching when possible. For example,

```
distance1 :: (Float,Float) -> (Float,Float) -> Float
distance1 (x,y) (x',y') = norm (x - x') (y - y')
```

is better than

```
distance2 :: (Float,Float) -> (Float,Float) -> Float
distance2 p p' = norm ((fst p) - (fst p')) ((snd p) - (snd p'))
```

9 Large Multi-Component Expressions

Carefully line up large multi-component expressions — like conditional expressions, guarded function definitions, case expressions, lists, tuples, records, and data types — that cross multiple lines. Here are some examples:

```
abs1 :: Float -> Float
abs1 x =
    if x < 0
    then -x
    else  x
```

```
abs2 :: Float -> Float
abs2 x =
    if x < 0
    then -x
    else  x
```

```
signs :: [Int]
signs =
    [ -1
    ,  0
    ,  1
    ]
```

```
data NobleGas
    = Helium
    | Neon
    | Argon
    | Krypton
    | Xenon
    | Radon
```